

«Talento Tech»

React JS

Clase 05



Clase N° 5 | Uso de useEffect y APIs

Índice

Clase N° 5 | Uso de useEffect y APIs

Introducción a useEffect

Ejemplo básico de uso en el código:

Simulando nuestra propia API

Carpeta public vs. Carpeta src

Creando nuestros datos

Probando nuestra API por consola

Desmenuzando el código

Integrando todo: useEffect + fetch + useState

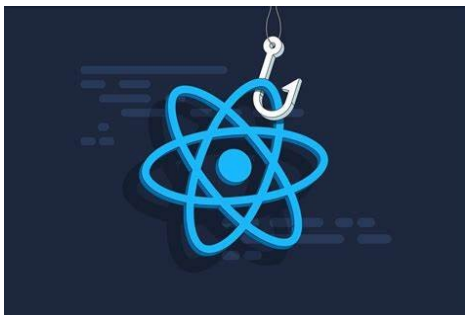
Explicación paso a paso:

Final

Objetivos de la Clase:

- Comprender qué son los efectos secundarios y cómo gestionarlos con el hook **useEffect**.
- Realizar peticiones a APIs para obtener datos dinámicamente.
- Manejar estados de carga y errores para mejorar la experiencia del usuario.

Introducción a useEffect



El Hook para todo lo "externo":

Imaginen que sus componentes de React son como una pizzería. El flujo principal es preparar la pizza con los ingredientes que ya tienen (las props y el estado). Pero, ¿qué pasa si se quedan sin muzzarella? Tienen que hacer algo "fuera" de ese flujo principal: llamar al proveedor.

Eso, en React, es un efecto secundario. Son operaciones que interactúan con el mundo exterior al componente

Ejemplos comunes de efectos secundarios:

- Pedir datos a una API.
- Manipular el DOM directamente.
- Poner un `setTimeout` o un `setInterval` (poner un temporizador).

Para manejar todo esto, React nos da el hook `useEffect`, que nos permite ejecutar un pedazo de código después de que el componente se renderiza en la pantalla.

La Sintaxis de `useEffect` es la siguiente:

🔥 Prestá atención a los números de los comentarios que nos va a ayudar a entenderlo mejor.

```
import React, { useEffect } from 'react';
useEffect(() => {
  // Código que queremos ejecutar (el efecto)
  console.log('El componente se acaba de mostrar en pantalla.');
```

```
  return () => {
    // La "limpieza" (opcional)
    console.log('El componente se va a desmontar. Limpiando...');
```

```
  };
}, [/*Array de dependencias */]);
```

Continúa con la explicación en la próxima hoja.

Desglosemos esto que parece más complicado de lo que es:

1. **La Función del Efecto:** Es el primer parámetro. Acá va la lógica que queremos ejecutar. Por ejemplo, el `fetch` para pedir datos.
2. **La función de limpieza (Opcional):** Es lo que `useEffect` retorna. Se ejecuta cuando el componente está por "desmontarse" (desaparecer de la pantalla) o antes de que el efecto se vuelva a ejecutar. Es ideal para cancelar suscripciones o temporizadores y evitar problemas de memoria.
3. **El array de dependencias (¡La Clave!):** Este array le dice a React cuándo tiene que volver a ejecutar el efecto
 - **[] (Array vacío):** El efecto de la `hooj` se ejecuta **una sola vez**, justo después del primer renderizado. Perfecto para buscar los datos iniciales de nuestra aplicación.
 - **[prop, estado]:** El efecto se ejecuta la primera vez y **cada vez que** el valor de prop o estado cambie.
 - **Si no ponés el array:** Cuidado con olvidartelos porque el efecto se ejecutará **después de cada renderizado**. Esto puede crear bucles infinitos y colgarte la aplicación. Casi nunca es usado.



Ejemplo básico de uso en el código:

Como anteriormente hicimos un contador con `useState`, en este caso vamos a usar `useEffect` para hacer algo simple con ese contador: mostrar por consola cuando el valor cambia. Eventualmente, la funcionalidad del log puede cambiar por otr mas compleja:

```
// En /componentes/EjemploUseEffect.jsx
import { useState, useEffect } from "react"; // Importamos los hooks juntos!
export function Contador() {
  const [contador, setContador] = useState(0);
  useEffect(() => {
    console.log("El componente se renderizó o el contador cambió.");
  }, [contador]); // 📌 depende del contador

  return (
    <div style={{ margin: "20px", padding: "15px", border: "1px solid black" }}>
      <h3>Ejemplo sencillo de useEffect</h3>
      <p>Valor actual: {contador}</p>
      <button onClick={() => setContador(contador + 1)}>Sumale +1</button>
      <button onClick={() => setContador(contador - 1)}>Restale -1</button>
    </div>
  );
}
```

);}

¿Que estamos viendo en nuestro código?

1. Tenemos un contador hecho con useState que empieza en 0.
2. Cada vez que hacemos clic en el botón (y de forma poco segura) sumamos uno a la variable contador o restamos según corresponda.
3. useEffect escucha el cambio en contador (porque está en el array de dependencias [contador]), y cuando eso pasa, ejecuta el console.log("mensaje")

Con esto vemos en la consola del navegador cómo useEffect responde a los cambios del estado.

Veamos ahora como podemos trabajar con useEffect para hacer nuestra página mas ágil.

Simulando nuestra propia API

Para un eCommerce, necesitamos mostrar productos, y esos productos no van a estar escritos directamente en el código. Tienen que venir de algún lado. Se podrían pedir a una API externa, pero a veces estas tienen algunas limitaciones.

Por eso, vamos a trabajar con un archivo JSON local como si fuera una API remota y vamos a obtener su información como si fuera una API externa, con fetch.



Carpeta public vs. Carpeta src

Esta diferencia es un concepto teórico fundamental.

- **La carpeta src (source/fuente):** Es el corazón de nuestra aplicación. Todo el código que escribimos (componentes, hooks, estilos) vive acá. Cuando "compilamos" el proyecto, herramientas como Vite o Create React App agarran todo lo de src, lo procesan, optimizan y lo "empaquetan" en archivos JavaScript y CSS que el navegador entiende. No podemos usar fetch para leer un archivo de src, porque para el navegador, esa estructura de carpetas no existe. Si queremos usar algo de src, lo tenemos que importar.
- **La carpeta public:** Es como la "vidriera" de nuestra tienda. Todo lo que pongamos acá (imágenes, fuentes, o nuestro archivo de datos) no se procesa ni se empaqueta. Simplemente se copia tal cual a la carpeta final del proyecto. Esto significa que podemos acceder a estos archivos directamente a través de una URL, como si fuera un sitio web normal.

src es para nuestro código fuente, public es para los "assets" estáticos.

Creando nuestros datos

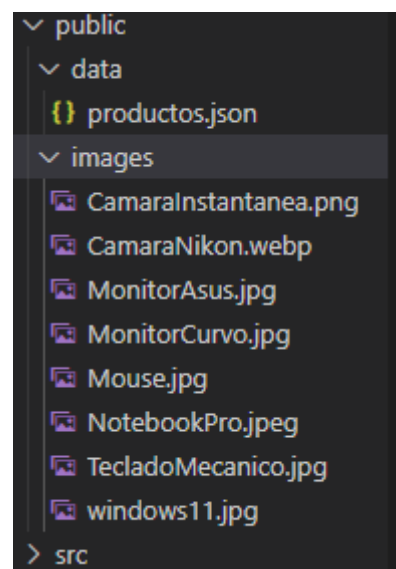
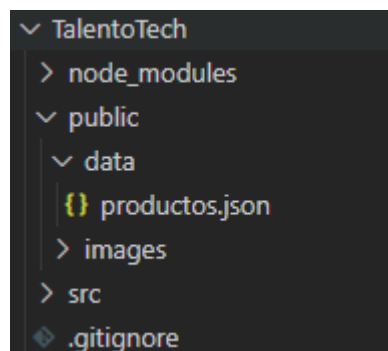
Cuando creamos el proyecto, vimos que teníamos una estructura de carpetas particular. Hasta el momento estuvimos trabajando en la carpeta `src`. La carpeta `Public` tiene una imagen ([vite.svg](#)) que es el ícono de la pestaña. Vamos a eliminar ese ícono y vamos a crear nuestros propios datos.

1. Dentro de la carpeta `public`, creen una nueva carpeta llamada `data`
2. Dentro de **`public/data`**, creen un archivo `productos.json`.
3. Dentro de la carpeta `public`, creen otra carpeta llamada `images`. Acá van a guardar las fotos de los productos.
Para arrancar, podemos descargar algunas imágenes de nuestro navegador, o descargar esta [carpeta](#).

Con las imágenes en nuestra carpeta, es momento de organizar nuestro archivo JSON para poder conectar nuestras imágenes con nuestra página. Así se vería nuestra estructura de carpetas:

Ahora, en `productos.json`, vamos a poner un array de objetos. Cada objeto es un producto:

```
[
  {
    "id": 1,
    "nombre": "Camara Instantánea",
    "precio": 85000,
    "stock": 22,
    "imagen": "/images/CamaraInstantanea.png"
  },
  {
    "id": 2,
    "nombre": "Camara Nikon",
    "precio": 97500,
    "stock": 2,
    "imagen": "/images/CamaraNikon.webp"
  },
  {
```



```

    "id": 3,
    "nombre": "Monitor Curvo",
    "precio": 110000,
    "stock": 2,
    "imagen": "/images/MonitorCurvo.jpg"
  },
]

```

Si querés agregar todos las imágenes de la carpeta images, recordá modificar la ruta y también el valor del ID de cada uno.

Importante: Nota que la ruta de la imagen empieza con una diagonal! ("/") Esto le dice al navegador que busque desde la raíz del sitio, que es nuestra carpeta public.

Probando nuestra API por consola

Vamos a recordar como usamos fetch para consultar una API y como podemos obtener por consola, la información que estamos necesitando. Este código lo podemos incorporar debajo del código existente en el archivo main.jsx del proyecto Vite.

La idea es simple: pedimos el archivo y, cuando llega la respuesta, la mostramos en la consola. Si no llega, mostramos un mensaje de error.

```

// src/main.jsx
fetch('/data/productos.json')
  .then(respuesta => {
    console.log('Respuesta cruda del servidor:', respuesta);
    return respuesta.json();
  })
  .then(datos => {
    console.log('¡Productos cargados!', datos);
  })
  .catch(error => {
    console.error('¡Ups! Hubo un error:', error);
  })
  .finally(() => {
    ;
  })

```

¿Cómo lo probás?

Para probar nuestro código vamos a seguir estos pasos:

1. Pegá ese código en tu main.jsx.
2. Asegurate de que tu archivo productos.json esté en public/data/productos.json.
3. Ejecutá tu proyecto (npm run dev).
4. Abrí las herramientas de desarrollador en tu navegador (con F12 o clic derecho > Inspeccionar) y andá a la pestaña Consola.

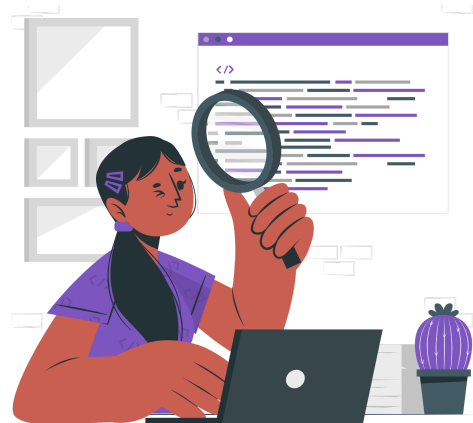
Deberías ver los mensajes y, finalmente, el array de tus productos. Se debería ver algo de esta manera:

```

Respuesta cruda del servidor: main.jsx:14
  Response {type: 'basic', url: 'http://localhost:5173/data/productos.json', re
    directed: false, status: 200, ok: true, ...}
¡Productos cargados! ▶ (7) [{...}, {...}, {...}, {...}, {...}, {...}, {...}] main.jsx:18
>
    
```

Desmenuzando el código

- **fetch('/data/productos.json')** : Le pide al servidor el archivo productos.json que está en la carpeta /data/. y nos devuelve una promesa que representa la respuesta del servidor
- **then(respuesta => { ... })** : Cuando el servidor contesta, entramos al primer .then. que nos devuelve un objeto de tipo Response con datos crudos (no JSON todavía).
- **console.log('Respuesta cruda del servidor:', respuesta); return respuesta.json();** : Una vez impreso por consulta la respuesta, la convierte en un objeto/array de JavaScript. (Esto también devuelve otra promesa).
- **then(datos => { ... })** : Ahora sí tenemos los datos ya convertidos en JSON y los mostramos en consola:
- **console.log('¡Productos cargados!', datos); catch(error => { ... })** : Si algo sale mal caemos en este bloque y al igual que los demás, lo mostramos por consola: **console.error('¡Ups! Hubo un error:', error);**
- **finally(() => {console.log('¡Productos cargados!', datos)})** Se ejecuta siempre al final, haya funcionado o no.



Integrando todo: useEffect + fetch + useState

Como vimos anteriormente, cuando pedimos datos, estos no aparecen al instante. Puede haber una demora o puede que la petición falle. Una buena aplicación debe manejar estas situaciones para que el usuario sepa qué está pasando.

Para esto, vamos a usar tres useState:

1. **productos:** Para guardar los datos cuando lleguen.
2. **cargando (loading):** Un booleano (true/false) que nos dice si estamos esperando la respuesta
3. **error:** Para guardar un mensaje de error si algo sale mal.

Ahora juntemos todo en un componente. Dentro de nuestra carpeta componentes, creamos una carpeta y su archivo .jsx llamada Productos.

En el vamos a escribir el siguiente componente:

```
import React, { useState, useEffect } from 'react';

function Productos({Mensaje}) {
  const [productos, setProductos] = useState([]);
  const [error, setError] = useState(null);
  const [cargando, setCargando] = useState(true);

  useEffect(() => {
    fetch('/data/productos.json')
      .then((respuesta) => {
        if (!respuesta.ok) {
          throw new Error('No se pudo cargar la información de los productos');
        }
        return respuesta.json();
      })
      .then((datos) => {
        setProductos(datos);
      })
      .catch((error) => {
        setError(error.message);
      })
      .finally(() => {
        setCargando(false);
      });
  }, []);
}
```

```

    if (cargando) {
      return <p>Cargando productos, por favor espere...</p>;
    }
    if (error) {
      return <p>Error: {error}</p>;
    }

    return (
      <div>
        <h1>{Mensaje}</h1>
        <ul>
          {productos.map((producto) => (
            <li key={producto.id}>
              <h2>{producto.nombre}</h2>
              <img src={producto.imagen} alt={producto.nombre} width="150" />
              <p>Precio: ${producto.precio}</p>
            </li>
          ))}
        </ul>
      </div>
    );
  }
}

export default Productos;

```

Con este if (cargando) y if (error), le damos al usuario una respuesta inmediata sobre el estado de la aplicación. ¡Eso es mejorar la experiencia de usuario (UX)!

Explicación paso a paso:

1. Importamos los hooks desde react y generamos la función con sus estados de inicio:

```

import React, { useState, useEffect } from 'react';

function Productos() {
  const [productos, setProductos] = useState([]);
  const [error, setError] = useState(null);
  const [cargando, setCargando] = useState(true);

```

- productos: lista de productos cargados (arranca vacía).
- error: mensaje de error en caso de fallo.
- cargando: indica si estamos esperando la respuesta (true al inicio).

2. Efecto con useEffect

```
useEffect(() => {
  fetch('/data/productos.json')
    .then((respuesta) => {
      if (!respuesta.ok) {
        throw new Error('No se pudo cargar la información de los productos');
      }
      return respuesta.json();
    })
    .then((datos) => {
      setProductos(datos);
    })
    .catch((error) => {
      setError(error.message);
    })
    .finally(() => {
      setCargando(false);
    });
}, []);
```

- [] al final: significa que este useEffect se ejecuta solo una vez, cuando el componente aparece en pantalla.
- explicación:
 1. Pide el archivo productos.json.
 2. Si la respuesta no es válida (!respuesta.ok), lanza un error.
 3. Si es correcta, convierte los datos a JSON y los guarda en productos.
 4. Si ocurre un error, lo guarda en error.
 5. Al final (exitoso o no), cambia cargando a false.

3. Renderizado condicional

```
if (cargando) {
  return <p>Cargando productos, por favor espere...</p>;
}
if (error) {
  return <p>Error: {error}</p>;
}
```

- Si estamos cargando, muestra un mensaje de “Cargando...”.
- Si hubo error, muestra el mensaje de error.

4. Mostramos los productos

```

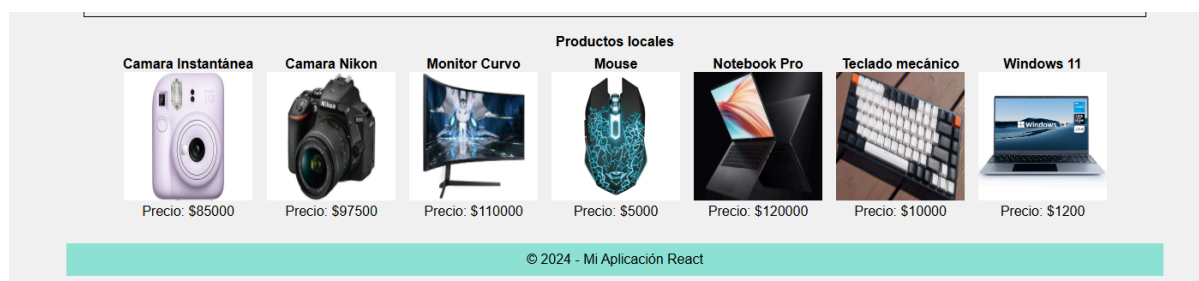
return (
  <div>
    <h1>{Mensaje}</h1>
    <ul>
      {productos.map((producto) => (
        <li key={producto.id}>
          <h2>{producto.nombre}</h2>
          <img src={producto.imagen} alt={producto.nombre} width="150"
height="150"/>
          <p>Precio: ${producto.precio}</p>
        </li>
      ))}
    </ul>
  </div>
);
    
```

- Mostramos el mensaje que mandamos como props según corresponda.
- Recorre el array productos con .map.
- Por cada producto, crea un mostrando su nombre, imagen y precio.
- key={producto.id} es importante en React para identificar cada elemento de la lista.

Final

Lo único que nos queda es importar y agregar el componente productos a nuestra aplicación.

Los imágenes guardados en nuestra carpeta images ya se ven correctamente en nuestra página. Te invito a encontrarle el estilo correspondiente para que las imágenes se muestren en forma horizontal.



Reflexión final

En esta clase, aprendimos a gestionar efectos secundarios con **useEffect**, realizar peticiones a APIs para cargar datos y manejar estados de carga y errores. También descubrimos cómo utilizar MockAPI para simular un backend, lo que nos permite trabajar con datos realistas sin necesidad de configurar un servidor.

Estas habilidades son esenciales para construir aplicaciones React que interactúen con datos externos, brindando una experiencia de usuario fluida y profesional. Al dominar estos conceptos, estarás preparado para desarrollar aplicaciones más completas y robustas.

TalentoLab - Proyecto Final



Misión: Vas a crear un directorio de contactos importantes simple. La aplicación va a cargar los datos de los creadores del sitio desde un archivo JSON local y los mostrará en pantalla, manejando de forma profesional los tiempos de carga y los posibles errores.

Directorio de contactos

Tu tarea es construir esta aplicación aplicando los conceptos de la clase:

- 1. La fuente de datos - nuestro archivo JSON:**
 - Dentro de la carpeta correspondiente creá un archivo `nosotros.json`.
 - Este archivo va a ser un array de objetos. Creá al menos 4 personas.. Cada usuario debe tener: `id`, `nombre`, `email`, `puesto` (ej: "CEO") y `foto`.
 - Guardá las fotos de perfil (podés usar avatares genéricos de internet) en la carpeta `public/images`. Recordá que la ruta en el JSON debe ser relativa a `public` (ej: `/images/avatar1.png`).
- 2. La Lógica de Carga - `useEffect` y `useState`:**
 - En el componente principal de tu aplicación, definí los tres estados que ya conocés:
 - `nosotros` (inicializado como `[]`)
 - `cargando` (inicializado como `true`)
 - `error` (inicializado como `null`)
 - Usá el hook `useEffect` para hacer la llamada `fetch` a tu archivo `/data/nosotros.json`.

3. La experiencia de usuario - Renderizado condicional:

- Tu aplicación debe ser informativa para el usuario en todo momento:
 - Si cargando es true, mostrá en pantalla un mensaje claro, como un "Cargando equipo...".
 - Si la llamada fetch falla y el estado error se actualiza, mostrá un mensaje de error visible.

4. La interfaz - mapeando los datos:

- Una vez que los datos estén en tu estado usuarios, usá .map() para crear una "tarjeta de contacto" por cada persona.
- Cada tarjeta debe mostrar la foto del usuario, su nombre, puesto y email.
- Organizá las tarjetas en una grilla para que se vea prolijo y ordenado y agregalos al principio del footer.

Pautas y consejos

- **Estructura de Componentes:** Podés crear un componente Directorio que maneje la lógica del fetch y los estados. Y dentro de él, podés mapear y renderizar un componente TarjetaContacto para cada usuario, pasándole los datos por props.
- **Estilos:** ¡Dale tu toque! Podés usar la página [Bootswatch](#) para buscar estilos o CSS puro o lo que te quede más cómodo para que las tarjetas de contacto se vean bien

Materiales y recursos adicionales:

- ★ [Documentación oficial de useEffect](#)
- ★ [Guía introductoria a las promesas en JavaScript](#)
- ★ [Estado y ciclo de vida en React](#)
- ★ [Cómo manejar errores en JavaScript](#)

Preguntas para reflexionar:

- ¿Qué diferencia a **useEffect** de otros hooks como **useState**?
- ¿Por qué es importante manejar estados de carga y errores al trabajar con datos de APIs?
- ¿Cómo podrías integrar múltiples peticiones a APIs en un solo componente sin afectar el rendimiento?

Próximos pasos:

- Construir un **formulario presentacional** simple y reutilizable.
- Manejar los eventos onChange y onSubmit para crear un **componente controlado**.
- Aplicar el patrón **Contenedor/Presentacional** para manejar la lógica del formulario.



Buenos Aires
aprende
Agencia de Habilidades para el Futuro

BA Buenos
Aires
Ciudad